

Parallel Football - G7 Report

Steven Galban, David Mihnea-Stefan, Vivian Xiao

February 2026

1 Introduction

In this project, we study the problem of algorithmic control in the Parallel Football game, where the objective is to maximize the number of footballs delivered to a team's home cell on a shared 32×32 grid. Each team controls multiple players that can move locally or kick footballs within a bounded Euclidean distance, while all teams act simultaneously and compete for the same resources. The problem combines elements of multi-agent coordination, spatial optimization, and adversarial interaction under strict movement and action constraints.

Our solution explores several strategic directions for addressing this challenge. We investigated greedy and distance-based heuristics that prioritize footballs based on their proximity to the home cell and the estimated number of kicks required to score. In parallel, we examined coordination strategies among players, including distributing players across the grid to reduce congestion and forming implicit passing chains to move footballs efficiently over longer distances. We also evaluated defensive and disruptive behaviors, such as positioning players near the home cell to secure incoming balls and limit opponents' access to high-value regions. Finally, we considered controlled randomization in decision-making to reduce predictability and mitigate conflicts arising from simultaneous actions. The remainder of this report details these approaches and analyzes their effectiveness.

2 Initial Insights and Observations

2.1 BasicPlayerMD

The `BasicPlayerMD` strategy represents our first complete iteration of a functional player controller and serves as a baseline for more advanced approaches developed later. This implementation follows a predominantly greedy paradigm, aiming to move footballs toward the home cell as efficiently as possible while enforcing simple coordination constraints between players.

Initial Placement. At the beginning of the game, player placement is handled by partitioning the 32×32 grid into four equal quadrants. Each player is assigned to a quadrant based on the team's home position and its player index, and then placed uniformly at random within that region. This strategy ensures an approximately even spatial distribution of players across the grid (and regions), reducing early congestion and allowing faster coverage of distant footballs. While the placement is randomized, the quadrant-based restriction prevents all players from clustering in the same area.

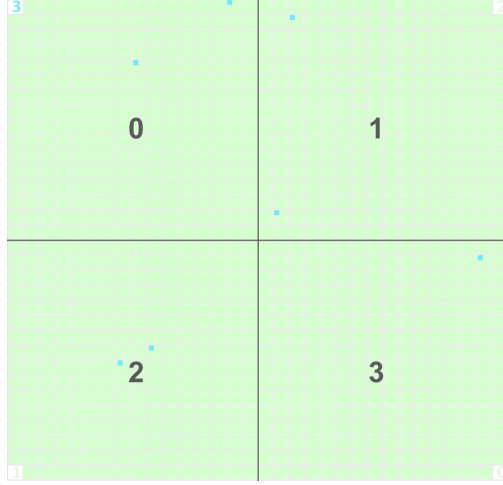


Figure 1: Partitioning of the grid into four quadrants and example of randomized initial player placement.

Move Decision Logic. During each turn, the algorithm distinguishes between two primary cases based on the contents of the player's current cell.

If a player is located on a cell containing at least one football, it attempts to kick the ball. Two subcases are considered. If the player is within kicking range K of the home cell (measured using Euclidean distance), the ball is kicked directly into the home cell, immediately scoring a point. Otherwise, the player kicks the ball as far as possible in the direction of the home cell. This is done by selecting a kick vector that maximizes progress toward the home while remaining within the circular radius constraint imposed by K . Diagonal directions are prioritized in order to maximize forward displacement per kick.

If the player is not standing on a football, it instead attempts to move toward the closest available ball. The target ball is selected by minimizing the Euclidean distance between the player and all football-containing cells on the board. In the event of a tie, preference is given to the ball whose direction is more closely aligned with the direction of the home cell, thereby biasing movement toward globally favorable positions.

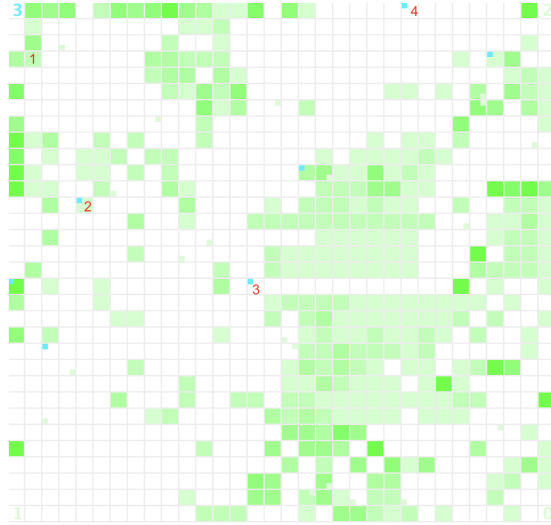


Figure 2: Kicking behavior when a player is on a football: direct scoring if within range K (Player 1), or maximal forward kick toward the home cell otherwise (Player 2). Movement decisions when no football is present on the current cell. Player 3 moves toward the nearest ball, while Player 4 resolves a tie by favoring the ball closer to the home cell.

Coordination Constraints. To reduce inefficiencies caused by player interference, two additional coordination constraints are enforced during move computation. First, no more than one player is allowed to target the same football in a given turn. Once a football is selected as a target, it is marked as considered and becomes unavailable to other players for that turn. Second, players are prevented from moving into the same destination cell, ensuring collision-free motion and avoiding wasted moves caused by overlapping trajectories.

Both constraints are implemented using auxiliary boolean grids that are updated incrementally as player actions are decided. One grid tracks occupied destination cells, while another tracks footballs that have already been assigned to a player. This mechanism enables lightweight coordination without explicit communication between players and preserves the overall greedy structure of the algorithm.

Observations. Although `BasicPlayerMD` is a simple greedy implementation with limited global planning, it demonstrated surprisingly strong performance. The strategy tends to underperform in the early stages of the game due to the randomized initial placement, but becomes increasingly effective in the later stages, when footballs accumulate near the home cell and can be cleared efficiently through short kicks and direct scoring.

```

1  int bestX = currx, bestY = curry;
2  int bestD2 = Integer.MAX_VALUE;
3  for (int x = 0; x < 32; x++) {
4      for (int y = 0; y < 32; y++) {
5          if (consideredBalls[x][y] && P <= _football.ballsLeft()) continue;
6          int dx = x - currx;
7          int dy = y - curry;
8          int d2 = dx * dx + dy * dy;
9
10         if (is_ball(x, y) &&
11             !occupiedCells[currx + get_sign(dx)][curry + get_sign(dy)]) {
12             if (d2 < bestD2) {
13                 bestD2 = d2;
14                 bestX = x;
15                 bestY = y;
16             } else if (d2 == bestD2 &&
17                 get_sign(dx) == get_sign(my_home_X - currx) &&
18                 get_sign(dy) == get_sign(my_home_Y - curry)) {
19                 bestX = x;
20                 bestY = y;
21             }
22         }
23     }
24 }

```

Listing 1: Coordination constraints preventing multiple players from targeting the same ball or cell

2.2 GreedyPlayer

While it exhibited strong performance, one subtle weakness observed in **BasicPlayerMD** was the players’ apparent aversion to diagonally located balls compared to more axis-aligned ones, even when the diagonal balls could be reached in fewer turns. This was addressed in the more-descriptively-named **GreedyPlayer**, which implemented the insights used in **BasicPlayerMD**, but utilized Chebychev distance, rather than Euclidean distance, when determining the distance between players and balls, which better modeled player movement. A more in-depth discussion of distance metrics is given in Section 4.1.

3 Strategies and Concepts

Given everything we have learned in the first two iterations of our player, we decided to combine the best of both strategies into a hybrid strategy (**HybridPlayer3**) based on (P, K) since it was difficult to find a single underlying strategy/scoring function that worked well across all configurations. In particular, we noticed

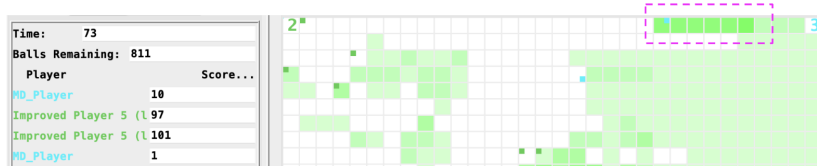


Figure 3: Game state during the early-to-mid phase of a match using **BasicPlayerMD**. Due to the randomized initial placement, the strategy initially exhibits slower progress. However, as footballs are progressively kicked toward the home cell, they accumulate in nearby regions, leading to efficient clearing and improved scoring performance in the later stages of the game.

that the initial greedy strategy seemed to do quite well when either P or K was small (not necessarily both), while a more global ball-assignment strategy (i.e. Group 1’s **ImprovedPlayer**) seemed to work better for larger P and K . Through running various trials at differing (P, K) combinations, we decided to select our greedy player (**GreedyPlayer3**) when $P \leq 10$ or $K \leq 2$, else select the global optimization player (**TwiceImprovedPlayer**).

The intuition behind this decision is simple:

“If the board is sparse or movement is restricted, just race. If the board is crowded and kicks matter, think strategically.”

3.1 TwiceImprovedPlayer

Our overall goal was simple: at every moment in the game, we want our players to be working on the balls that will give us the most points in the shortest amount of time, while minimizing wasted movement and avoiding unnecessary competition with opponents.

Rather than treating all balls equally, we wanted our players to make intelligent decisions about:

- Which balls are actually worth pursuing,
- Which player should go to which ball,
- When to race opponents, and when to ignore them,
- How to move balls efficiently back toward home.

ImprovedPlayer: Value-Based Ball Selection

The original **ImprovedPlayer** from Group 1 followed a global, value-based strategy. At each time step, it evaluated every ball on the board and assigned it a “desirability score.” This score captured several intuitive ideas:

- Balls that are closer to our home square are better, since they require fewer kicks to score.

- Stacked balls (multiple balls in the same cell) are more valuable, since one trip can score multiple points.
- Balls in our own quadrant are slightly preferable, since we can usually reach them sooner than opponents.

After ranking all balls by desirability, the strategy assigned players to the highest-value opportunities while trying to avoid duplication (i.e., not sending multiple players to the same ball).

When a player reached a ball, it attempted to kick it directly home if possible. Otherwise, it kicked the ball closer to home, ideally toward a teammate to enable a relay-style progression.

TwiceImprovedPlayer: Scaling for Different Game Regimes

While the original approach worked well, we observed that different configurations of (P, K) fundamentally change how the game behaves.

For example:

- When there are few players, races are more isolated and local.
- When there are many players, congestion becomes important.
- When the kick radius is small, movement dominates the game.
- When the kick radius is large, long relay chains become possible.

We therefore modified the strategy to adapt to these regimes.

1. Better Distance Modeling. Since diagonal movement is allowed, we adjusted how distance was conceptually measured so that it better reflected how many steps it actually takes to reach a target. This made player movement decisions more accurate and realistic.

2. Long-Chain Awareness. In high-player settings, we found that balls requiring many kicks to reach home were often inefficient to pursue early. Too many players would cluster, leading to congestion and wasted turns.

To address this, we increased the effective “cost” of balls that were far from home in crowded settings. Conceptually, this meant:

“In crowded games, prioritize quick wins over long, risky chains.”

3. Smarter Competition Handling. Rather than simply avoiding balls that opponents were closer to, we adjusted how strongly competition mattered depending on how valuable the ball was. High-value balls were worth contesting; low-value balls were not.

4. Relay-Friendly Kicking. When a ball could not be scored immediately, players aimed to move it closer to home in a way that teammates could continue the progression. This encouraged implicit relay behavior without explicitly forming rigid bucket-brigade roles.

3.2 GreedyPlayer3

While our previous greedy heuristic implementations generally seemed to work well and perform quite robustly, they were not without their weaknesses, some of which were very obvious when watching a game play out. Tie-breaking techniques present in this lineage of player classes since `BasicPlayerMD` proved to be effective, encouraging certain behaviors only in the case of a tie appeared to be insufficient in many cases. For example:

- Because players kick balls towards their home cell, they'd often create a "moat" around it. Since players target balls based on distance, this means our players would move farther and farther away from the goal, leaving all the balls nearby unattended.
- Our greedy players completely ignored the presence or actions of players from opposing teams and they'd often target balls on cells already occupied by other players. This would end up wasting their time, as the ball would certainly be kicked somewhere else by the time they reached the cell.
- Players saw the number of balls on a given cell as a binary: either a cell had a ball, or it didn't. This meant that a player might target a cell with 1 ball that's 10 units away, rather than a cell with 100 balls that was 11 units away, losing many potential points to other teams.

In theory, these issues could all be addressed separately by introducing explicit behavior, but in addition to increasing the complexity of our implementations, that could have harmed the otherwise robust performance we found by honing in on the greedy heuristic. That's why we decided to incorporate a score function, similar to the one used by `ImprovedPlayer`, as described above. The idea would be that, instead of choosing the cell with a ball that minimized distance to the current player, the player would instead target the cell the maximized some score, which could depend on a number of other metrics in addition to distance. That way, our players could still be greedy, but we could have more control over the metric optimized on and introduce more balanced, nuanced movement. Using an iterative approach, we could find which metrics our players would benefit the most from, and how to combine them into a single, coherent, easy-to-reason-about function.

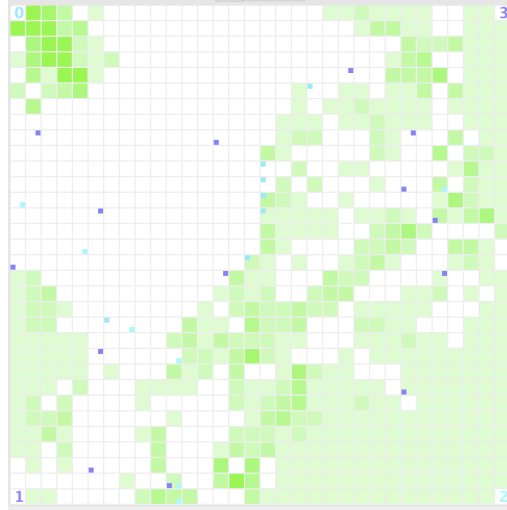


Figure 4: A notable problem with early **GreedyPlayer** iterations. In many games, players would kick balls near the goal, but no player would follow through and kick them into the home cell. This would create a ”moat” of width K around the home cell, surrounding an unattended pile of balls that enemy players would often steal.

4 Implementation

4.1 TwiceImprovedPlayer

TwiceImprovedPlayer is a refined version of Group 1’s **ImprovedPlayer** that keeps the same overall template—(1) evaluate ball cells, (2) assign players to targets, (3) move/kick—but makes several key changes to improve robustness across different (P, K) regimes. In particular, we (i) use distance measures that better reflect diagonal movement, (ii) explicitly scale long-kick-chain cost as P grows, and (iii) incorporate a stronger and more “value-aware” notion of competition.

Per-turn computation overview

Each turn, the player computes the move for all P agents simultaneously:

1. Cache current board state and all teammate/opponent positions.
2. Construct a list of *ball cells* (grid positions with at least one ball) and precompute properties such as distance-to-home and estimated number of kicks to score.
3. For any teammate already standing on a ball cell, immediately decide a kick action (direct-to-home if possible, otherwise to an intermediate target).

4. For remaining teammates, compute a score for each (player, ball) pair and greedily assign players to distinct balls by descending score.
5. Emit movement vectors for assigned targets (one-step motion, allowing diagonals), and $(0,0)$ kick vectors for movers.

Ball features and base desirability

For each cell c with $B(c) > 0$ balls, we estimate how difficult it is to score from that location. Let h denote our home square. We compute:

$$d_H(c) = \|c - h\|_2, \quad \text{ kicksToScore}(c) = \left\lceil \frac{d_H(c)}{K} \right\rceil.$$

This estimate is intentionally simple: it approximates the number of length- K kicks required if we always kick directly toward home.

We also apply a mild territorial preference by boosting balls located in our quadrant:

$$q(c) = \begin{cases} 1.5 & \text{if } c \text{ is in our quadrant} \\ 1.0 & \text{otherwise.} \end{cases}$$

The resulting “intrinsic value” of a ball cell is:

$$V(c) = B(c) \cdot q(c).$$

Intuitively, $V(c)$ is high when a cell has many balls and is likely to be contestable by our team earlier than opponents.

Movement distance model (Chebyshev)

Players are allowed to move one step in x and one step in y per turn, so diagonal movement is as fast as axis-aligned movement. Therefore the number of turns to reach a target is better approximated by Chebyshev distance:

$$d_\infty(p, c) = \max(|p_x - c_x|, |p_y - c_y|).$$

We use d_∞ for player-to-ball distances in assignment and movement. (We continue to use Euclidean distance for kick feasibility because kicks are limited by an Euclidean radius.)

Player–ball assignment score

The core improvement is the score used to decide *which player should pursue which ball*. For a player at position p and ball cell c , we define an effective cost

$$\eta(p, c) = d_\infty(p, c) + \lambda(P) \cdot \text{ kicksToScore}(c),$$

where $\lambda(P)$ increases with P . This reflects the observation that in high-player regimes, long kick-chains are harder to execute reliably due to congestion and interference. Empirically, we used:

$$\lambda(P) = 0.15 \left(\frac{P}{7} \right)^{2.2},$$

which keeps the kick-chain term small near $P = 7$ while making it much more dominant for large P (e.g., $P = 30$).

The base score is then:

$$base(p, c) = \frac{V(c)}{\eta(p, c) + 1}.$$

This prioritizes ball-dense cells and de-prioritizes targets that are far away or require many kicks to finish.

Competition penalty. We also compute the closest opponent distance to the same ball cell:

$$d_{\infty}^{opp}(c) = \min_{o \in Opponents} d_{\infty}(o, c).$$

If an opponent is strictly closer than us ($d_{\infty}^{opp}(c) < d_{\infty}(p, c)$), then the ball is likely to be “stolen” before we arrive. We penalize proportionally to how far behind we are:

$$penalty(p, c) = \begin{cases} \alpha \cdot (d_{\infty}(p, c) - d_{\infty}^{opp}(c) + 1) & \text{if } d_{\infty}^{opp}(c) < d_{\infty}(p, c) \\ 0 & \text{otherwise.} \end{cases}$$

In our implementation, α was set around 1.5.

Finally, we make competition matter *more* on high-value opportunities by scaling the penalty by the base value:

$$score(p, c) = base(p, c) - \beta \cdot penalty(p, c) \cdot base(p, c).$$

This has two desirable effects:

- Low-value balls get ignored if contested (no reason to fight for scraps).
- High-value balls are still attractive, but we avoid them when we are significantly behind.

```

1 double quadrantBonus = isInOurQuadrant(ball.position) ? 1.5 : 1.0;
2
3 // lambda(P): scale long kick-chains up as P increases
4 double ksMult = 0.15 * Math.pow(P / 7.0, 2.2); // e.g. P=7 -> 0.15, P=30 -> ~4
5
6 double eta = dist + ksMult * ball.kicksToScore;
7 double base = (ball.count * quadrantBonus) / (eta + 1.0);
8
9 // If opponent is closer, penalize proportional to how far behind we are.
10 double competitionPenalty = 0.0;
11 if (minOppDist < dist) {
12     competitionPenalty = 1.5 * (dist - minOppDist + 1);
13 }
14
15 // Make competition matter more on high-value opportunities.
16 double score = base - 0.25 * competitionPenalty * base;

```

Listing 2: Assignment scoring function used by `TwiceImprovedPlayer`.

Greedy assignment procedure

Computing the globally optimal assignment is expensive. Instead, we compute scores for all (player, ball) pairs, sort them by score, and greedily select the best pairs while ensuring:

- each player is assigned to at most one ball, and
- each ball cell is assigned to at most one player.

This produces good results in practice, and is computationally cheap for a 32×32 grid.

Kicking behavior and intermediate targets

If a player is currently on a ball cell, it does not move; it kicks.

Direct score. If the home cell is within kick range, we kick directly:

$$\text{if } \|p - h\|_2 \leq K, \quad \Delta = h - p.$$

Intermediate kick. Otherwise, we choose a landing cell t within Euclidean distance K that maximizes a heuristic:

- **Progress:** the reduction in Euclidean distance-to-home from p to t ,
- **Relay bonus:** landing near a teammate (to continue the chain),
- **Safety:** avoiding landing near opponents (to prevent immediate steals),

- **Range usage:** a mild preference for using a large fraction of the kick range.

We implement this by iterating over all candidate landing cells t on the board, checking feasibility ($\|t - p\|_2 \leq K$), and selecting the maximum-scoring target.

```

1 if (euclideanDistance(pos, homeSquare) <= K) {
2     kx[playerIndex] = homeSquare.x - pos.x;
3     ky[playerIndex] = homeSquare.y - pos.y;
4     return;
5 }
6 Point bestTarget = findBestKickTarget(pos, opponentPositions);
7 kx[playerIndex] = bestTarget.x - pos.x;
8 ky[playerIndex] = bestTarget.y - pos.y;

```

Listing 3: Kick decision: direct-to-home when possible, else best intermediate target.

Complexity considerations

Each turn, **TwiceImprovedPlayer** scans the board to find ball cells ($O(32^2)$), computes opponent positions ($O(P \cdot \#teams)$), and computes assignment scores for remaining players and balls. In the worst case where most cells contain balls, the score matrix is $O(P \cdot 32^2)$, but in practice the number of ball cells is much smaller than 1024. The intermediate kick search iterates over all board cells for each kicking player, yielding $O(\#kickers \cdot 32^2)$ worst-case. Despite these scans, runtime remained comfortably within limits in our experiments due to the small fixed grid size.

In summary, **TwiceImprovedPlayer** preserves the strong global allocation structure of **ImprovedPlayer**, while improving its behavior in crowded regimes by (i) modeling travel time more accurately with Chebyshev distance, (ii) scaling long-chain risk with P , and (iii) applying a value-aware competition penalty to reduce wasted races.

4.2 GreedyPlayer3

An intermediate greedy player implementation, **GreedyPlayer2**, attempted to solve the aforementioned "moat" problem by introducing a simple scoring function that retained the distance-to-player metric, but added a small weight to balls closer to the home square.

```
1 int distToBall = chebychevDistance(curPoint, position);
2 int distBallToHome = chebychevDistance(curPoint, home);
3 double score = distToBall + (distBallToHome * 0.25);
4 if (score < bestScore) {
5     bestScore = score;
6     best = curPoint;
7 }
```

Listing 4: The scoring function used by **GreedyPlayer2**. Each turn, this score would be computed for each ball-containing cell for each player. The player will then move towards the cell with the lowest score.

While this helped alleviate the problem to some extent, the extra pull introduced by the weighting proved insufficient, and the issue generally remained. Moreover, it utilized chebychev distance when determining the distance from a ball to the home square, which, considering balls are generally kicked over a certain euclidean distance (i.e. K), would put too much weight on balls located diagonally from the home square.

GreedyPlayer3 was written to adhere to this same basic strategy, but using a more complex and more nuanced score function:

```

1 private double calculateScore(Point cell, Point playerPosition) {
2     if (!inBounds(cell) || cell.equals(home)) {
3         return 0;
4     }
5
6     // Encourage closer balls and cells with more balls
7     int density = numberOfBalls(cell) + (kickTargetCells.contains(cell) ? 1 : 0);
8     if (density == 0) {
9         return 0;
10    }
11    int playerDistance = chebychevDistance(cell, playerPosition);
12    double homeDistance = cell.distance(home);
13    double score = Math.sqrt(density) / playerDistance;
14
15    // Encourage cells closer to home, especially with scoreable balls
16    if (homeDistance < kickDistance()) {
17        score *= 2;
18    } else {
19        score += 1 / homeDistance;
20    }
21
22    // Avoid cells with other players
23    int playersInCell = getPlayersInCell(cell);
24    if (numberOfBalls(cell) < playersInCell) {
25        score *= 0.01;
26    } else {
27        score *= Math.pow(0.75, playersInCell);
28    }
29
30    return score;
31 }

```

Listing 5: The scoring function used by **GreedyPlayer3**. Unlike the one used by **GreedyPlayer2**, players will target the cell that maximizes this function.

Determining the exact shape of this score function involved a mix of intuition, and trial-and-error. Every change made was proceeded by a series of tournaments against other teams' most recent player implementations to determine if that particular change caused a non-negligible increase in average score. The resulting score function:

- Incorporates distance, encouraging players to target nearby balls as before
- For the first time in our lineage of greedy players, considers the number of balls on a cell, encouraging players to target cells with a higher density. This metric has diminishing returns to avoid overwhelming the distance metric.

- Adds a small amount of weight to cells that a teammate is expected to kick to, slightly encouraging emergent "passing" behavior.
- Adds weight to cells closer to the home cell, especially if they're within kicking distance. This effectively solved the aforementioned "moat" problem.
- Discourages players from targeting cells that contain players from other teams, especially if there are more players than balls.

The result is a player that is much more aware of the state of the board than past players. In head-to-head tournaments, **GreedyPlayer3** outperforms **GreedyPlayer** and **GreedyPlayer2** by approximately 17% and 12% respectively, in terms of average score.

5 Results

We evaluate our final submission, **HybridPlayer3**, on the official tournament across 10 configurations of (P, K) . Our goal in designing a hybrid was not to optimize for a single regime, but to maintain strong performance across very different game dynamics (sparse vs. crowded boards, local races vs. long kick chains). Overall, the tournament results support this motivation: our player placed in the top two in 9 out of 10 configurations, and placed third in the remaining configuration.

5.1 Summary of Tournament Performance

Table 1 summarizes our rank and key statistics across all configurations. We report *AvgScore* as the primary metric (average points per game), and also include *Delta* and *Epsilon* as measures of how decisively we win and how competitive we remain even in games we do not win.

Sub-player	P	K	Rank	AvgRank	AvgScore	(Delta, Epsilon)
GreedyPlayer3	1	15	2	1.651	306.200	(33.300, 17.598)
GreedyPlayer3	3	3	1	1.401	321.383	(54.452, 11.908)
GreedyPlayer3	4	32	2	1.665	301.611	(30.998, 15.351)
GreedyPlayer3	7	6	1	1.554	309.062	(40.396, 13.460)
GreedyPlayer3	7	18	1	1.659	311.712	(42.980, 13.731)
GreedyPlayer3	10	10	2	1.935	303.769	(30.499, 17.824)
GreedyPlayer3	46	1	1	1.584	398.939	(127.630, 16.749)
TwiceImprovedPlayer	30	3	2	1.456	325.390	(58.425, 11.404)
TwiceImprovedPlayer	30	20	2	1.610	328.158	(59.473, 13.454)
TwiceImprovedPlayer	255	32	3	1.991	304.430	(55.523, -6.298)

Table 1: End-of-project tournament summary for **HybridPlayer3**.

Are our results “good” relative to our goals? Yes: the hybrid decision rule achieved the main objective we set out in the Approach section—strong performance across diverse regimes without committing to a single scoring function. Across most settings, we maintain both (1) a strong average score and (2) a top average rank, indicating that we are not overfit to a niche configuration.

What do Delta and Epsilon tell us? We observed two consistent patterns: (1) our *Delta* values are large in nearly every configuration, indicating that when we win, we tend to win by a meaningful margin; and (2) our *Epsilon* values are positive in 9 out of 10 configurations, meaning that even when we fail to place first, we are typically still scoring above the game-average (i.e., we are losing to another top-tier strategy rather than collapsing below the lobby baseline). The one exception is $(P, K) = (255, 32)$, where our negative epsilon suggests that losses in this extreme configuration are comparatively less competitive.

5.2 Analysis: Why the Hybrid Worked

A key insight from development was that the game behaves qualitatively differently depending on (P, K) .

GreedyPlayer3 regimes (7/10 configurations). When the board is sparse (small P) or kick interactions are local (small K), simple racing behaviors and local decision-making dominate. In these settings, **GreedyPlayer3** is frequently the best-performing strategy, placing first in four configurations: $(3, 3)$, $(7, 6)$, $(7, 18)$, and $(46, 1)$. The most extreme example is $(46, 1)$, where **GreedyPlayer3** achieves an AvgScore of 398.939 with an exceptionally large Delta of 127.630, reflecting both strong point accumulation and decisive wins.

TwiceImprovedPlayer regimes (3/10 configurations). When both P and K are large, congestion and multi-step kick chains become more important. In these settings, our **TwiceImprovedPlayer** remains highly competitive: it places second in both $(30, 3)$ and $(30, 20)$, with AvgScore above 325 in both cases. However, the extreme $(255, 32)$ setting represents a failure mode for our approach. While our AvgScore remains respectable, the drop to third and the negative epsilon suggest that the dynamics at this scale require a more specialized high- P strategy (e.g., stronger collision handling, more explicit area control, or a different approach to ball valuation).

5.3 Head-to-Head Comparison with Other Groups

To better understand our performance relative to specific strategies, we analyzed the head-to-head (H2H) win-loss records from the tournament logs. These records reveal how often our player obtained a higher score than a specific opponent in the same match.

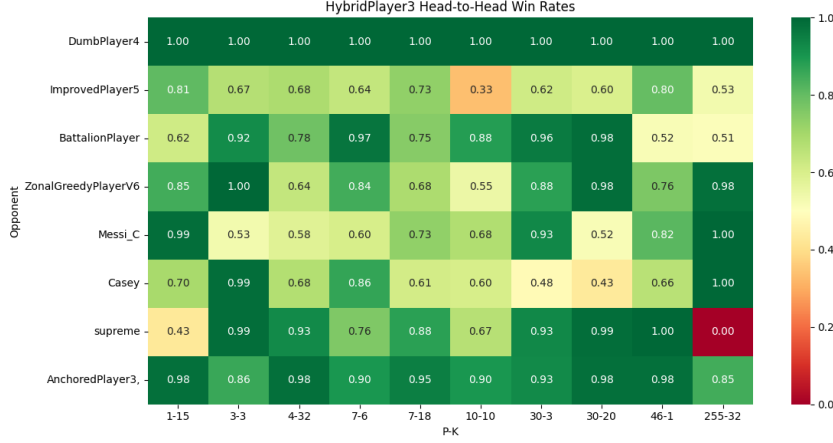


Figure 5: Head-to-Head Win Rates of HybridPlayer3 against all opponents across different (P, K) configurations. Green indicates a winning record (> 0.5), while Red indicates a losing record. HybridPlayer3 only had losing H2Hs against 3 different players in 5 configurations: ImprovedPlayer5 (10,10), Casey (30,3) and (30,20), and supreme (1,15) and (255,32).

Dominance in the Greedy Regime. In configurations where our GreedyPlayer3 logic was active ($P \leq 10$ or $K \leq 2$), we consistently dominated the field.

- **vs. The Field:** In the (46,1) configuration, we achieved a win rate of $> 50\%$ against every single opponent, including a solid 51.5% win rate against the second-place BattalionPlayer. This confirms that our local density heuristics manage congestion better than competitors.
- **The “Unlucky” 2nd Place:** In the (4,32) tournament, we placed 2nd overall behind ZonalGreedyPlayerV6. However, the H2H analysis reveals that we actually **beat** ZonalGreedyPlayerV6 in direct matchups, winning 64% of our mutual games. This suggests that while their strategy was slightly better at gaining points in lobbies with weaker bots, our player was superior in direct competition.

Rivalries in the TwiceImproved Regime. In high- P games ($P \geq 30$) where TwiceImprovedPlayer was active, our primary rival was Casey.

- **vs. Casey:** In the (30,3) configuration, we placed 2nd behind Casey, but the H2H record was close (48% wins vs. 50% losses). However, as kick distance increased to (30,20), Casey pulled ahead (43% win rate), suggesting their long-range coordination handles interference better.

The “Supreme” Anomaly. The team supreme presented a unique case. In sparse games like (1,15), they defeated us decisively (43% win rate), likely

due to a highly optimized movement heuristic. Conversely, in dense games like (3, 3) or (4, 32), we completely reversed this, achieving win rates of 67% and 63% respectively. The exception was the saturated (255, 32) game, where **supreme** achieved a 100% win rate against us (and everyone else!), confirming that our assignment logic worsens under maximum density while their static formation thrives.

Our strongest competitive advantage is consistency: across most configurations, our strategy stays at or near the top rather than excelling in only a single regime.

We also stood out most clearly in the “local race” regime. In (46, 1) our AvgScore is substantially higher than the rest of the field, suggesting that our greedy scoring function and opponent-avoidance heuristics are especially effective when kicks are limited and movement efficiency dominates.

In contrast, in mid-to-large kick regimes such as (30, 3) and (30, 20), the top of the leaderboard is more crowded, and our performance is best characterized as “strong but narrowly second.” This reinforces the motivation behind the hybrid: rather than overfitting a single global strategy, we aimed for stability across the full evaluation suite.

5.4 Takeaways

The main lesson from the tournament is that robust heuristics can outperform more complex planning when the environment is adversarial and highly stochastic due to simultaneous action. Our best-performing configurations were those where the game rewarded simple, repeatable efficiency (short races, local kicks, and avoiding wasted contested moves). Our worst-performing configuration highlights a clear future direction: extreme high- P and high- K regimes likely require a fundamentally different coordination model than either greedy racing or value-based assignment.

6 Contributions

6.1 Impact

A key contribution of our group to the class was reframing how the Parallel Football problem should be approached in practice. In the early stages of the project, many teams proposed highly sophisticated strategies that appeared strong on paper but were often unstable when deployed in a fully adversarial and simultaneous setting. These approaches frequently struggled with unpredictability, interference from other teams, and sensitivity to small changes in game state.

In contrast, our work demonstrated that a simple, well-structured baseline could achieve strong and reliable performance. By focusing on greedy, interpretable heuristics combined with minimal coordination constraints, our algorithm consistently performed well across a wide range of scenarios. This out-

come challenged the prevailing assumption that increasingly complex logic was necessary to succeed in the game.

As a result, our approach influenced the overall direction of the project within the class. Several teams adopted similar ideas directly, others incorporated our algorithm as a subroutine within more complex strategies, and some teams used our player as a benchmark or training target, tuning parameters through empirical methods such as regression based on match outcomes. The success of this baseline highlighted the importance of robustness and empirical validation over purely theoretical complexity.

More broadly, we believe our contribution emphasized an important methodological lesson: in complex multi-agent systems, starting with a standard, stable solution and iteratively improving it based on observed performance can be more effective than immediately pursuing advanced or highly specialized techniques. This perspective shaped both our own subsequent iterations and the strategies explored by other teams throughout the project.

6.2 Individual Contributions

Throughout the project, all team members were actively involved in brainstorming sessions and strategic discussions. We collaborated closely when analyzing game dynamics, identifying weaknesses in early approaches, and proposing alternative heuristics. Design decisions were generally made through group discussion, with ideas iteratively refined based on empirical observations and experimental results. This collaborative process played a key role in shaping both the baseline strategy and its subsequent improvements.

Mihnea David was primarily responsible for the design and implementation of the initial baseline player, `BasicPlayerMD`, but also `RelayFirst`, `GreedyAssist` and `GreedyAssistLee`. His work focused on defining the core algorithmic structure, including quadrant-based player placement, greedy movement toward footballs using Euclidean distance, and distance-aware kicking heuristics that maximize forward progress toward the home cell. He also implemented coordination constraints to prevent multiple players from targeting the same football or occupying the same destination cell. In addition, he explored multiple heuristic variants aimed at maximizing scoring efficiency and evaluated their performance through extensive testing. These efforts established a robust and interpretable baseline that strongly influenced the overall direction of the project.

Vivian Xiao was primarily responsible for the creation of the `HybridPlayer` strategies, including creating the intermediate `GreedyPlayer2` and the final `TwiceImprovedPlayer`. In particular, she designed an improved scoring mechanism for deciding which cells to assign to which players, as described in section 4.1. She also ran various tournaments across different (P, K) configurations in order to find the optimal thresholds (i.e. when to use `GreedyPlayer3` versus `TwiceImprovedPlayer` and the coefficients for the scoring functions. She also made a large improvement to `TwiceImprovedPlayer` specifically for small P , which added the heuristic of number of balls within distance K away from the

goal to encourage players to finish off balls.

Steven Galban introduced some of the high-level philosophies that guided the group throughout the process, including preferring simple solutions wherever possible, and relying heavily on empirical data rather than solely on theory or intuition (with the rationale that the problem is too stochastic and unpredictable in nature to do otherwise). In addition to introducing the concept of a greedy heuristic (which would be implemented through David’s **BasicPlayerMD**) and the importance of using the correct distance metrics, he also implemented **GreedyPlayer** and **GreedyPlayer3**, experimenting with various parameters to create a well-performing score function.

7 Future Directions and Limitations

One limitation of our current approach lies in the incomplete exploitation of coordinated team play. In particular, we explored an extension we refer to as *PlayerAssist*, which was designed to encourage collaboration between teammates when a direct kick to the home cell was not feasible. The core idea was that, instead of always prioritizing individual progress, a player could choose to kick the ball toward a nearby teammate, enabling continuity of play and potentially faster scoring through multi-step coordination.

We implemented this mechanism using a weighted heuristic that favored passing to a teammate under certain conditions, for example when the teammate was within a large fraction (e.g., at least 0.9) of the player’s maximum kicking range and positioned closer to the home cell. While this strategy is theoretically appealing and aligns well with cooperative multi-agent behavior, it performed poorly in practice. The main reason appears to be the sensitivity of the approach to the chosen weights and thresholds. Suboptimal tuning often led to inefficient ball circulation or missed scoring opportunities. With more time, we would have liked to perform a more systematic exploration of the weight space or learn these parameters automatically.

Another limitation is that our player model does not sufficiently account for opponents’ future actions. While **GreedyPlayer3** does discourage targeting cells currently occupied by players from opposing teams, ideally we would also penalize cells that may soon be occupied by enemy players, and deeply reason about adversarial intent or contestation over specific balls. This becomes especially problematic when assigning objectives to players (e.g., targeting a specific ball), since an opponent may already be closer or more likely to reach that ball first. Incorporating more explicit opponent modeling or contest-aware heuristics could significantly improve decision quality in such scenarios.

Moreover, we believe that the **GreedyPlayer3** scoring function has additional room for improvement. We experimented incorporating multiple additional metrics not found in the final implementation, including:

- The score of adjacent cells
- Proximity to teammates

- Distance to opposing players
- Proximity to other teams' home cells

While these all seem like reasonable adjustments, we were unable to incorporate them into the score function in a way that resulted in empirical improvement. However, with some extra time and further tuning, we believe these metrics could be used to increase performance further.

However, despite the proven strength of the greedy heuristic, we believe that the approach with the highest theoretical ceiling would involve training a policy via reinforcement learning, potentially using a neural network. Such an approach could, in theory, learn advanced coordination techniques and plan around opponent behavior in a way that would be difficult, if not impossible, to encode in a score function. Implementing a reinforcement learning solution that could outperform our current solution, however, would require a substantial amount of effort in selecting features, modeling opponents, and tuning hyperparameters, as well as a potentially prohibitive amount of computational resources, so we decided against pursuing this strategy. Nonetheless, given an indefinite amount of time to craft a solution with the highest possible performance, we would likely adopt an approach involving machine learning.

8 Acknowledgments

We would like to give a special shout out to Group 1 (Vince Tiu, Maya Huizar, Aeshon Balasubramanian) for inspiring our own **TwiceImprovedPlayer** and for helping us move towards a more score/formula-oriented approach, which paid off in the end

ChatGPT/Gemini were utilized to help with formulating certain subsections of this Work and formatting in LaTeX, including text, tables, graphs, and code.

9 Conclusion

Overall, this was a very intriguing project and all three of us greatly enjoyed working on the problem! We liked how open-ended the problem was (i.e. there is no feasible way given our time frame to completely explore all combinations of P,K and train against all types of players). As a result, this gave us and the rest of the teams free reign to try whatever strategies we wanted to and watch the tournaments unfold. The trial games during each of the intermediate classes were also extremely interesting and useful, as most of our modifications in between classes was due to the player dynamics/interactions that we observed.

As for improvements, we believe that Parallel Football is already a well-designed problem and codebase. The constraints are clearly defined, and the codebase allowed us to fully focus on creating the player logic rather than interacting with the GUI. In terms of running the tournaments, perhaps there could be some more explanation in the beginning regarding the Tournament metrics

(i.e. H2H metrics) and register methods. The exact mechanics surrounding the timing of the register method invocations and where player state should be initialized was often unclear and occasionally resulted in unexpected bugs and crashes.